```
import * as Notifications from 'expo-
notifications';
import * as TaskManager from 'expo-task-
manager';
import { Platform } from 'react-native';
import { getSettings, getPending,
savePending } from './storage';
import { escalateIfNotTaken } from
'./emergency';
import { sendReport } from './report';

const BG_TASK = 'DAWAEI_BG_NOTIFICATION';

Notifications.setNotificationHandler({
  handleNotification: async () => ({
shouldShowAlert: true, shouldPlaySound:
true, shouldSetBadge: false }),
});

// يشتغل لما يوصل إشعار "التصعيد" والتطبيق بالخلفية
(أندرويد)
TaskManager.defineTask(BG_TASK, async ({
data }) => {
  const content =
data?.notification?.request?.content ??
data?.notification?.data;
  const d = content?.data ?? data?.data ??
{};
  if (d.kind === 'escalate') await
escalateIfNotTaken(d.doseId);
```

```
});
Notifications.registerTaskAsync(BG_TASK).ca
tch(() => {});

export async function setupNotifications()
{
  const { status } = await
Notifications.requestPermissionsAsync();
  if (Platform.OS === 'android') {
    await
Notifications.setNotificationChannelAsync('
meds', {
      name: 'مواعيد الدواء',
      importance:
Notifications.AndroidImportance.MAX,
      sound: 'default',
      vibrationPattern: [0, 500, 300, 500],
      lockscreenVisibility:
Notifications.AndroidNotificationVisibility
.PUBLIC,
    });
  }
  await
Notifications.setNotificationCategoryAsync(
'dose', [
    { identifier: 'TAKEN', buttonTitle: '✅
أخذت الدواء', options: { opensAppToForeground:
false } },
  ]);
  return status === 'granted';
}

// جدولة تذكير يومي + إشعار تصعيد بعد دقائق الانتظار
export async function scheduleMed(med) {
```

```
  const { waitMinutes } = await
getSettings();
  const [h, m] =
med.time.split(':').map(Number);
  const esc = new Date(2000, 0, 1, h, m +
(waitMinutes || 10));

  const reminderId = await
Notifications.scheduleNotificationAsync({
    content: {
      title: 'وقت الدواء 💊',
      body: `${med.name} ${med.dose ? '- '
+ med.dose : ''}`,
      categoryIdentifier: 'dose',
      data: { kind: 'remind', medId:
med.id, name: med.name, time: med.time },
      sound: 'default',
      ...(Platform.OS === 'android' ? {
channelId: 'meds' } : {}),
    },
    trigger: { hour: h, minute: m, repeats:
true },
  });

  const escalateId = await
Notifications.scheduleNotificationAsync({
    content: {
      title: 'لم يتم تأكيد الدواء',
      body: 'جاري الاتصال برقم الطوارئ',
      data: { kind: 'escalate', medId:
med.id, doseId: `${med.id}` },
      ...(Platform.OS === 'android' ? {
channelId: 'meds' } : {}),
    },
```

```
    trigger: { hour: esc.getHours(),
minute: esc.getMinutes(), repeats: true },
  });
  return { reminderId, escalateId };
}

export async function cancelMed(med) {
  for (const id of [med.reminderId,
med.escalateId]) if (id) await
Notifications.cancelScheduledNotificationAs
ync(id);
}

// تسجيل أن الجرعة انأخذت
export async function markTaken(med) {
  const pending = await getPending();
  pending[med.id] = { name: med.name, time:
med.time, taken: true, at: Date.now() };
  await savePending(pending);
  sendReport({ type: 'taken', dose:
pending[med.id] }).catch(() => {});
}

// يفتح التطبيق من الإشعار
export function listenToResponses(meds) {
  const sub =
Notifications.addNotificationResponseReceiv
edListener(async (r) => {
    const d =
r.notification.request.content.data || {};
    const med = meds.find((x) => x.id ===
d.medId);
    if (!med) return;
    if (r.actionIdentifier === 'TAKEN')
```

```
await markTaken(med);
    else if (d.kind === 'remind') {
      const pending = await getPending();
      pending[med.id] = { name: med.name,
time: med.time, taken: false };
      await savePending(pending);
    } else if (d.kind === 'escalate') await
escalateIfNotTaken(d.doseId);
  });
  // لما يوصل إشعار التذكير نسجّل أنه بانتظار الرد
  const recv =
Notifications.addNotificationReceivedListen
er(async (n) => {
    const d = n.request.content.data || {};
    if (d.kind === 'remind') {
      const pending = await getPending();
      pending[d.medId] = { name: d.name,
time: d.time, taken: false };
      await savePending(pending);
    } else if (d.kind === 'escalate') await
escalateIfNotTaken(d.doseId);
  });
  return () => { sub.remove();
recv.remove(); };
}
```